Practical No.: 02

NAME: Bhalerao Tanvi Vitthal
ROLLNO.: 08
COURSE: AI&DS
CLASS: BE
SUB: Computer Laboratory-IV
Title: Multiclass Classification using Convolutional Neural Networks (CNN).

0.1 Import Required Libraries

[1]:
```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense,␣
 ↳Dropout
from tensorflow.keras.utils import to_categorical

from sklearn.metrics import confusion_matrix
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

0.2 Load the MNIST Dataset

[2]:
```
(x_train, y_train), (x_test, y_test) = mnist.load_data()

print("Training data shape:", x_train.shape)
print("Testing data shape:", x_test.shape)
```

```
Training data shape: (60000, 28, 28)
Testing data shape: (10000, 28, 28)
```

0.3 Data Preprocessing

[3]:
```
# Normalize pixel values
x_train = x_train / 255.0
x_test = x_test / 255.0

# Reshape for CNN (adding channel dimension)
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

# One-hot encode labels
y_train_cat = to_categorical(y_train, 10)
```

```
y_test_cat = to_categorical(y_test, 10)
```

0.4 Build CNN Model

[4]:
```
model = Sequential()

model.add(Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)))
model.add(MaxPooling2D((2,2)))

model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPooling2D((2,2)))

model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))

model.add(Dense(10, activation='softmax'))

model.summary()
```

D:\Old ssd\Anaconda3\Anaconda\Lib\site-packages\keras\src\layers\convolutional\base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)

Trainable params: 225,034 (879.04 KB)

Non-trainable params: 0 (0.00 B)

0.5 Compile the Model

[5]:
```
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

0.6 Train the Model

[6]:
```
history = model.fit(
    x_train,
    y_train_cat,
    epochs=10,
    batch_size=128,
    validation_split=0.1
)
```

```
Epoch 1/10
422/422                 19s 41ms/step -
accuracy: 0.9019 - loss: 0.3234 - val_accuracy: 0.9825 - val_loss: 0.0637
Epoch 2/10
422/422                 19s 44ms/step -
accuracy: 0.9686 - loss: 0.1045 - val_accuracy: 0.9868 - val_loss: 0.0465
Epoch 3/10
422/422                 17s 40ms/step -
accuracy: 0.9759 - loss: 0.0820 - val_accuracy: 0.9877 - val_loss: 0.0426
Epoch 4/10
422/422                 21s 40ms/step -
accuracy: 0.9812 - loss: 0.0634 - val_accuracy: 0.9887 - val_loss: 0.0388
Epoch 5/10
422/422                 18s 42ms/step -
accuracy: 0.9844 - loss: 0.0532 - val_accuracy: 0.9900 - val_loss: 0.0355
Epoch 6/10
422/422                 20s 40ms/step -
accuracy: 0.9859 - loss: 0.0464 - val_accuracy: 0.9905 - val_loss: 0.0352
Epoch 7/10
422/422                 18s 42ms/step -
```

```
accuracy: 0.9878 - loss: 0.0410 - val_accuracy: 0.9882 - val_loss: 0.0396
Epoch 8/10
422/422                 21s 44ms/step -
accuracy: 0.9890 - loss: 0.0361 - val_accuracy: 0.9925 - val_loss: 0.0335
Epoch 9/10
422/422                 16s 38ms/step -
accuracy: 0.9896 - loss: 0.0322 - val_accuracy: 0.9922 - val_loss: 0.0340
Epoch 10/10
422/422                 17s 40ms/step -
accuracy: 0.9909 - loss: 0.0300 - val_accuracy: 0.9918 - val_loss: 0.0319
```

0.7 Evaluate the Model

[7]:
```
test_loss, test_accuracy = model.evaluate(x_test, y_test_cat)
print("Test Accuracy:", test_accuracy)
```

```
313/313                 2s 5ms/step -
accuracy: 0.9927 - loss: 0.0238
Test Accuracy: 0.9926999807357788
```

0.8 Predictions

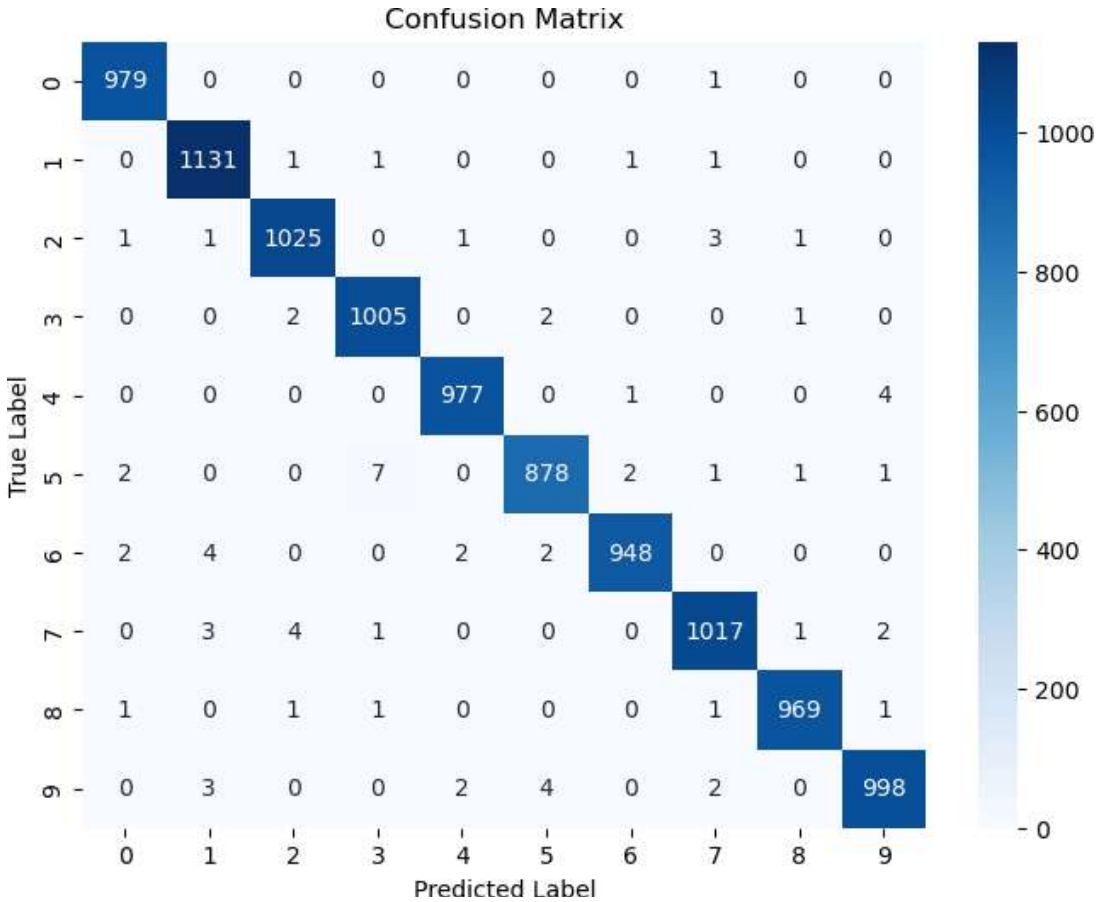
[8]:
```
y_pred = model.predict(x_test)
y_pred_classes = np.argmax(y_pred, axis=1)
```

```
313/313                 2s 5ms/step
```

0.9 Confusion Matrix

[9]:
```
cm = confusion_matrix(y_test, y_pred_classes)

plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix")
plt.show()
```


Confusion Matrix
True Label
Predicted Label
0 1 2 3 4 5 6 7 8 9
979 0 0 0 0 0 0 1 0 0
0 1131 1 1 0 0 1 1 0 0
1 1 1025 0 1 0 0 3 1 0
0 0 2 1005 0 2 0 0 1 0
0 0 0 0 977 0 1 0 0 4
2 0 0 7 0 878 2 1 1 1
2 4 0 0 2 2 948 0 0 0
0 3 4 1 0 0 0 1017 1 2
1 0 1 1 0 0 0 1 969 1
0 3 0 0 2 4 0 2 0 998
1000
800
600
400
200
0

[ ]: